Security Audit Report

Milestone 2 | Security-Hardened Retrieval-Augmented Generation System

Prepared for: DATA 790 - Advanced AI Frameworks in Production **Student:** William McDonald

Assessment date: October 26, 2026 **Evidence:** Offline unit/adversarial tests; source-code review of the uploaded Milestone 1 notebook

1. Executive summary

Milestone 2 extends the existing course-topic RAG baseline with an input-security gateway, retrieval-time source screening and authorization, PII redaction at input/retrieval/output, privacy-conscious audit events, and automated adversarial regression tests. **98 of 98 local deterministic tests passed; 0 failed and 0 were skipped.** No live model-backed prompt-injection tests were performed as part of this build. These results do not establish production safety or compliance with data-protection law.

2. Assessed assets and scope

In scope: one Jupyter-based RAG workflow, the Python policy module, retrieved-context screens, synthetic athlete records, application-side document class rules, output screening, and offline pytest evidence. The existing source corpus is ten course-focused knowledge sections.

Out of scope: real athlete or patient records; institutional SSO; hosting/network controls; multi-user isolation; live UNC AI Gateway penetration tests; formal legal/compliance review; external security scanning; and independent red-team assessment.

3. Trust boundaries

Boundary	Enforcement / behavior
User -> query	Reject malformed/control-character/HTML inputs; screen known direct and encoded injection; redact detectable PII before embeddings or prompts.
Vector store -> evidence	Filter permitted classification at query time and check each returned document again; quarantine known injection text and unsafe metadata.
Evidence -> model	Present inspected source JSON as untrusted context in a user message; system message forbids treating sources as instructions.
Model -> user	Redact detectable PII and selected secrets; refuse obvious instruction-like unsafe model responses.
Actor -> access	Access is determined in application code; demo role is not authenticated. Missing document classification is denied.

4. Threat model and controls

ID / threat	Attack scenario / potential impact	Implemented safeguard and status
T1: Direct injection	User tries to override system policy or reveal privileged instructions.	Bounded keyword/phrase rules, Unicode checks, source isolation. **Tested offline**.
T2: Indirect / RAG poisoning	Poisoned retrieved page instructs the model to ignore the user task or exfiltrate data.	Quarantine screened source text/metadata and pass only inspected data. **Tested offline**.
T3: Sensitive data leakage	Email, phone, ID, card or API token travels into model context or output.	Detectors and category-specific redaction at three boundaries. **Tested on synthetic strings**.
T4: Unauthorized document	Coach/public user retrieves a private team or synthetic medical note.	Classification filtering + Python ACL; restricted requires trainer and explicit document allowlist. **Policy tested offline**.
T5: Output misuse	Generated answer repeats a malicious instruction or recognizable secret.	Post-generation scan, redaction and controlled refusal. **Tested offline**.
T6: Abuse / resource limits	Very long user inputs or oversized retrieved text exhaust resources.	Caps on questions, source text and context chunk count. No service-level rate limits. **Partially tested**.

5. Vulnerabilities identified in Milestone 1 and changes

• **Minimal user-input screening:** Milestone 1 checked length, empty strings, and a small regex denylist. Updated to normalized, typed, bounded input inspection and known encoded-pattern detection.

• **No retrieved-document safeguards:** Milestone 1 directly built model context from Chroma results. Updated to document authorization, poisoned-source quarantine, metadata checks, and retrieval redaction.

• **No dedicated PII or output protection:** Milestone 1 returned generation results directly. Updated to common-format redaction, output screening and non-content-bearing audit events.

• **Limited security evidence:** Milestone 1 contained four simple security prompts. Updated with repeatable offline automated adversarial, PII, permission and false-positive tests.

6. Measured offline security testing

The included Python regression suite executes against deterministic gateway functions and mock retrieved records. It does not call an embedding service or chat model. Case expectations are fixed prior to the test run; the resulting JSON includes counts and category totals, not raw prompts.

Control/test group	Cases	Passed	Failed
Input validation / malformed content	12	12	0
Direct + Base64/hex injection rules	22	22	0
Indirect / document poisoning	12	12	0
Synthetic PII redaction	22	22	0
Application-side document access	14	14	0
Legitimate-query acceptance	10	10	0
Output / audit / integration checks	6	6	0
TOTAL	**98**	**98**	**0**

Observed selected-case pass rate: 98/98 = 100.0% (offline, known cases only). This does **not** estimate open-world attack detection rate or LLM attack success rate. The finite benign set also does not establish population-wide false-positive rates.

7. Security test limitations

• No adversarial calls to the configured UNC AI Gateway were executed; live output resistance, quality and token overhead remain unmeasured.

• Rule-based injection screening cannot detect every semantic, multilingual, typoglycemic, nested-encoding, or multimodal attack.

• PII patterns omit many identifiers and contextual medical information. Use dedicated regulated-data review before production.

• Synthetic access controls assume trusted upstream identity and source classification; they are not an identity-management system.

• Stored Chroma collection is ephemeral and the demonstration does not implement per-user rate limiting, tenant isolation or operational incident response.

8. OWASP LLM Top 10 risk-to-control mapping

Mapping is to 2025 OWASP LLM risk names. These are **design alignments, not certification or proof of complete mitigation**.

OWASP category	Milestone 2 measure / remaining gap
LLM01 - Prompt Injection	Question and source checks; quarantines and output screening. Novel/model-level attacks remain.
LLM02 - Sensitive Information Disclosure	Synthetic PII/secrets redaction and minimal audit logging. No real PHI compliance validation.
LLM04 - Data and Model Poisoning	Known malicious retrieved text quarantined. No independent third-party corpus provenance validation.
LLM05 - Improper Output Handling	Output controls for text, but no comprehensive HTML/SQL/UI sink controls.
LLM07 - System Prompt Leakage	Specific extraction patterns and refusals; no guarantees against alternative prompts.
LLM08 - Vector and Embedding Weaknesses	Metadata classification and double ACL check; no enterprise multi-tenant vector-store controls.
LLM10 - Unbounded Consumption	String/source/chunk limits; rate limiting, quotas and spending alerts not implemented.

9. Priority remediation plan

• **Before grading/submission:** run the full notebook with the owner's valid credentials and record real safe-query and attack outcomes; verify the end-to-end source provenance and re-run baseline quality/cost metrics.

• **Before any deployment:** integrate trusted campus identity, centrally managed permissions, logging and incident response, quotas, persistent vector-store security, and human review of private data access.

• **Before regulated datasets:** seek formal institutional review; replace synthetic demonstrations with approved workflows, dedicated PII/PHI controls and independently evaluated defenses.

10. Evidence, reproducibility and references

Source files: `william_mcdonald_milestone_2_security_hardened.ipynb`, `milestone2_security/pipeline.py`, `tests/test_security_controls.py`, `run_security_eval.py`, `reports/security_test_results.json`. To reproduce offline counts run `python run_security_eval.py` from the project root. To regenerate this document run `python generate_audit_pdf.py`.

References: OWASP LLM Prompt Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html and OWASP LLM Top 10: https://genai.owasp.org/llm-top-10/

Audit conclusion: A tested educational baseline has been implemented. Live evaluation, authorization integration and broader assurance remain open work; the system should not be characterized as production secure.